

Chapter 11. Signals

Signals were introduced by the first Unix systems to allow interactions between User Mode processes; the kernel also uses them to notify processes of system events. Signals have been around for 30 years with only minor changes.

The first sections of this chapter examine in detail how signals are handled by the Linux kernel, then we discuss the system calls that allow processes to exchange signals.

11.1. The Role of Signals

A signal is a very short message that may be sent to a process or a group of processes. The only information given to the process is usually a number identifying the signal; there is no room in standard signals for arguments, a message, or other accompanying information.

A set of macros whose names start with the prefix `SIG` is used to identify signals; we have already made a few references to them in previous chapters. For instance, the `SIGCHLD` macro was mentioned in the section "[The `clone\(\)`, `fork\(\)`, and `vfork\(\)` System Calls](#)" in [Chapter 3](#). This macro, which expands into the value 17 in Linux, yields the identifier of the signal that is sent to a parent process when a child stops or terminates. The `SIGSEGV` macro, which expands into the value 11, was mentioned in the section "[Page Fault Exception Handler](#)" in [Chapter 9](#); it yields the identifier of the signal that is sent to a process when it makes an invalid memory reference.

Signals serve two main purposes:

- To make a process aware that a specific event has occurred
- To cause a process to execute a signal handler function included in its code

Of course, the two purposes are not mutually exclusive, because often a process must react to some event by executing a specific routine.

[Table 11-1](#) lists the first 31 signals handled by Linux 2.6 for the 80x86 architecture (some signal numbers, such as those associated with `SIGCHLD` or `SIGSTOP`, are architecture-dependent; furthermore, some signals such as `SIGSTKFLT` are defined only for specific architectures). The meanings of the default actions are described in the next section.

Table 11-1. The first 31 signals in Linux/i386

#	Signal name	Default action	Comment	POSIX
1	SIGHUP	Terminate	Hang up controlling terminal or process	Yes
2	SIGINT	Terminate	Interrupt from keyboard	Yes

2

3	SIGQUIT	Dump	Quit from keyboard	Yes
4	SIGILL	Dump	Illegal instruction	Yes
5	SIGTRAP	Dump	Breakpoint for debugging	No
6	SIGABRT	Dump	Abnormal termination	Yes
6	SIGIOT	Dump	Equivalent to SIGABRT	No
7	SIGBUS	Dump	Bus error	No
8	SIGFPE	Dump	Floating-point exception	Yes
9	SIGKILL	Terminate	Forced-process termination	Yes
10	SIGUSR1	Terminate	Available to processes	Yes
11	SIGSEGV	Dump	Invalid memory reference	Yes
12	SIGUSR2	Terminate	Available to processes	Yes
13	SIGPIPE	Terminate	Write to pipe with no readers	Yes
14	SIGALRM	Terminate	Real-timer clock	Yes
15	SIGTERM	Terminate	Process termination	Yes
16	SIGSTKFLT	Terminate	Coprocessor stack error	No
17	SIGCHLD	Ignore	Child process stopped or terminated, or got signal if traced	Yes
18	SIGCONT	Continue	Resume execution, if stopped	Yes
19	SIGSTOP	Stop	Stop process execution	Yes
20	SIGTSTP	Stop	Stop process issued from tty	Yes
21	SIGTTIN	Stop	Background process requires input	Yes
22	SIGTTOU	Stop	Background process requires output	Yes
23	SIGURG	Ignore	Urgent condition on socket	No
24	SIGXCPU	Dump	CPU time limit exceeded	No
25	SIGXFSZ	Dump	File size limit exceeded	No
26	SIGVTALRM	Terminate	Virtual timer clock	No
27	SIGPROF	Terminate	Profile timer clock	No

2

28	SIGWINCH	Ignore	Window resizing	No
29	SIGIO	Terminate	I/O now possible	No
29	SIGPOLL	Terminate	Equivalent to SIGIO	No
30	SIGPWR	Terminate	Power supply failure	No
31	SIGSYS	Dump	Bad system call	No
31	SIGUNUSED	Dump	Equivalent to SIGSYS	No

Besides the regular signals described in this table, the POSIX standard has introduced a new class of signals denoted as real-time signals ; their signal numbers range from 32 to 64 on Linux. They mainly differ from regular signals because they are always queued so that multiple signals sent will be received. On the other hand, regular signals of the same kind are not queued: if a regular signal is sent many times in a row, just one of them is delivered to the receiving process. Although the Linux kernel does not use real-time signals, it fully supports the POSIX standard by means of several specific system calls.

A number of system calls allow programmers to send signals and determine how their processes respond to the signals they receive. [Table 11-2](#) summarizes these calls; their behavior is described in detail in the later section "[System Calls Related to Signal Handling](#)."

Table 11-2. The most significant system calls related to signals

System call

Description

`kill( )`

Send a signal to a thread group

`tkill( )`

Send a signal to a process

`tgkill( )`

Send a signal to a process in a specific thread group

`sigaction( )`

Change the action associated with a signal

`signal( )`

Similar to `sigaction( )`

`sigpending( )`

Check whether there are pending signals

`sigprocmask( )`

Modify the set of blocked signals

```
sigsuspend( )
```

Wait for a signal

```
rt_sigaction( )
```

Change the action associated with a real-time signal

```
rt_sigpending( )
```

Check whether there are pending real-time signals

```
rt_sigprocmask( )
```

Modify the set of blocked real-time signals

```
rt_sigqueueinfo( )
```

Send a real-time signal to a thread group

```
rt_sigsuspend( )
```

Wait for a real-time signal

```
rt_sigtimedwait( )
```

Similar to `rt_sigsuspend( )`

An important characteristic of signals is that they may be sent at any time to a process whose state is usually unpredictable. Signals sent to a process that is not currently executing must be saved by the kernel until that process resumes execution. Blocking a signal (described later) requires that delivery of the signal be held off until it is later unblocked, which exacerbates the problem of signals being raised before they can be delivered.

Therefore, the kernel distinguishes two different phases related to signal transmission:

Signal generation

The kernel updates a data structure of the destination process to represent that a new signal has been sent.

Signal delivery

The kernel forces the destination process to react to the signal by changing its execution state, by starting the execution of a specified signal handler, or both.

Each signal generated can be delivered once, at most. Signals are consumable resources: once they have been delivered, all process descriptor information that refers to their previous existence is canceled.

Signals that have been generated but not yet delivered are called pending signals . At any time, only one

pending signal of a given type may exist for a process; additional pending signals of the same type to the same process are not queued but simply discarded. Real-time signals are different, though: there can be several pending signals of the same type.

In general, a signal may remain pending for an unpredictable amount of time. The following factors must be taken into consideration:

- Signals are usually delivered only to the currently running process (that is, to the `current` process).
- Signals of a given type may be selectively blocked by a process (see the later section "[Modifying the Set of Blocked Signals](#)"). In this case, the process does not receive the signal until it removes the block.
- When a process executes a signal-handler function, it usually masks the corresponding signal i.e., it automatically blocks the signal until the handler terminates. A signal handler therefore cannot be interrupted by another occurrence of the handled signal, and the function doesn't need to be reentrant.

Although the notion of signals is intuitive, the kernel implementation is rather complex. The kernel must:

- Remember which signals are blocked by each process.
- When switching from Kernel Mode to User Mode, check whether a signal for a process has arrived. This happens at almost every timer interrupt (roughly every millisecond).
- Determine whether the signal can be ignored. This happens when all of the following conditions are fulfilled:
 - ◆ The destination process is not traced by another process (the `PT_PTRACED` flag in the process descriptor `ptrace` field is equal to 0).^[*]

[*] If a process receives a signal while it is being traced, the kernel stops the process and notifies the tracing process by sending a `SIGCHLD` signal to it. The tracing process may, in turn, resume execution of the traced process by means of a `SIGCONT` signal.
 - ◆ The signal is not blocked by the destination process.
 - ◆ The signal is being ignored by the destination process (either because the process explicitly ignored it or because the process did not change the default action of the signal and that action is "ignore").
- Handle the signal, which may require switching the process to a handler function at any point during its execution and restoring the original execution context after the function returns.

Moreover, Linux must take into account the different semantics for signals adopted by BSD and System V ; furthermore, it must comply with the rather cumbersome POSIX requirements.

11.1.1. Actions Performed upon Delivering a Signal

There are three ways in which a process can respond to a signal:

1. Explicitly ignore the signal.
2. Execute the default action associated with the signal (see [Table 11-1](#)). This action, which is predefined by the kernel, depends on the signal type and may be any one of the following:

Terminate

The process is terminated (killed).

Dump

The process is terminated (killed) and a `core` file containing its execution context is created, if possible; this file may be used for debug purposes.

Ignore

The signal is ignored.

Stop

The process is stopped i.e., put in the `TASK_STOPPED` state (see the section "[Process State](#)" in [Chapter 3](#)).

Continue

If the process was stopped (`TASK_STOPPED`), it is put into the `TASK_RUNNING` state.

3. Catch the signal by invoking a corresponding signal-handler function.

Notice that blocking a signal is different from ignoring it. A signal is not delivered as long as it is blocked; it is delivered only after it has been unblocked. An ignored signal is always delivered, and there is no further action.

The `SIGKILL` and `SIGSTOP` signals cannot be ignored, caught, or blocked, and their default actions must always be executed. Therefore, `SIGKILL` and `SIGSTOP` allow a user with appropriate privileges to terminate and to stop, respectively, every process,^[*] regardless of the defenses taken by the program it is executing.

[*] There are two exceptions: it is not possible to send a signal to process 0 (swapper), and signals sent to process 1 (init) are always discarded unless they are caught. Therefore, process 0 never dies, while process 1 dies only when the init program terminates.

A signal is fatal for a given process if delivering the signal causes the kernel to kill the process. The `SIGKILL` signal is always fatal; moreover, each signal whose default action is "Terminate" and which is not caught by a process is also fatal for that process. Notice, however, that a signal caught by a process and whose corresponding signal-handler function terminates the process is not fatal, because the process chose to terminate itself rather than being killed by the kernel.

11.1.2. POSIX Signals and Multithreaded Applications

The POSIX 1003.1 standard has some stringent requirements for signal handling of multithreaded applications:

- Signal handlers must be shared among all threads of a multithreaded application; however, each thread must have its own mask of pending and blocked signals.
- The `kill( )` and `sigqueue( )` POSIX library functions (see the later section "[System Calls Related to Signal Handling](#)") must send signals to whole multithreaded applications, not to a specific thread. The same holds for all signals (such as `SIGCHLD`, `SIGINT`, or `SIGQUIT`) generated by the kernel.
- Each signal sent to a multithreaded application will be delivered to just one thread, which is arbitrarily chosen by the kernel among the threads that are not blocking that signal.

- If a fatal signal is sent to a multithreaded application, the kernel will kill all threads of the application not just the thread to which the signal has been delivered.

In order to comply with the POSIX standard, the Linux 2.6 kernel implements a multithreaded application as a set of lightweight processes belonging to the same thread group (see the section "Processes, Lightweight Processes, and Threads" in Chapter 3).

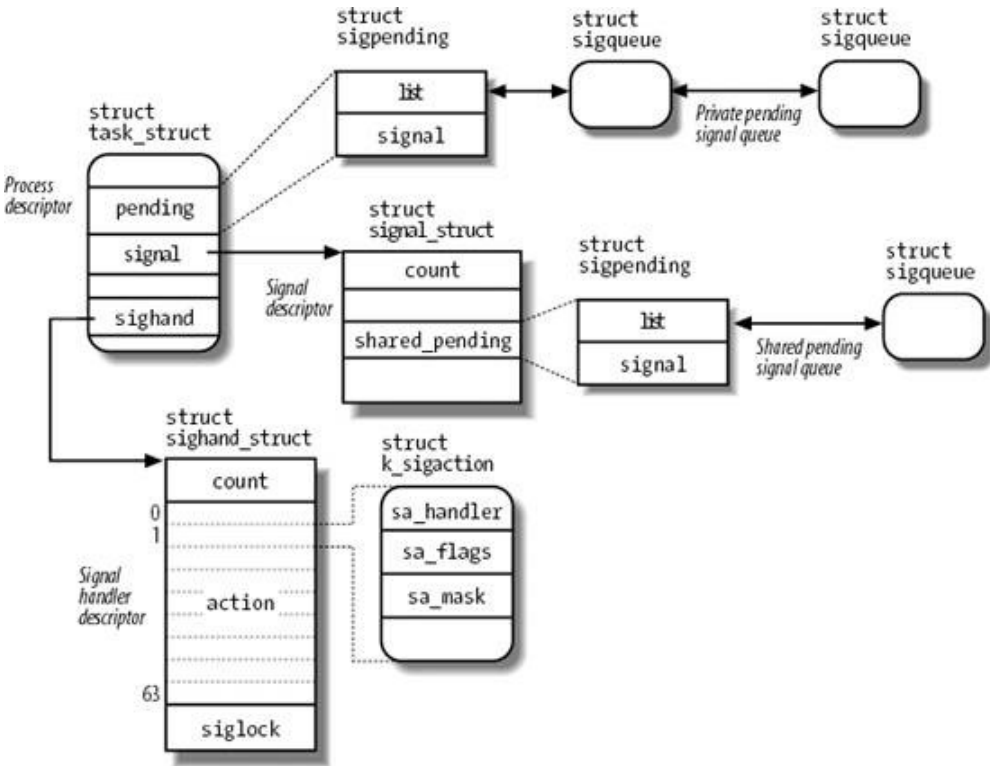
In this chapter the term "thread group" denotes any thread group, even if it is composed by a single (conventional) process. For instance, when we state that `kill( )` can send a signal to a thread group, we imply that this system call can send a signal to a conventional process, too. We will use the term "process" to denote either a conventional process or a lightweight process that is, a specific member of a thread group.

Furthermore, a pending signal is private if it has been sent to a specific process; it is shared if it has been sent to a whole thread group.

11.1.3. Data Structures Associated with Signals

For each process in the system, the kernel must keep track of what signals are currently pending or masked; the kernel must also keep track of how every thread group is supposed to handle every signal. To do this, the kernel uses several data structures accessible from the process descriptor. The most significant ones are shown in Figure 11-1.

Figure 11-1. The most significant data structures related to signal handling



The fields of the process descriptor related to signal handling are listed in Table 11-3.

Table 11-3. Process descriptor fields related to signal handling

Type

Name

Description

`struct signal_struct *``signal`

Pointer to the process's signal descriptor

`struct sighand_struct *``sighand`

Pointer to the process's signal handler descriptor

`sigset_t``blocked`

Mask of blocked signals

`sigset_t``real_blocked`Temporary mask of blocked signals (used by the `rt_sigtimedwait( )` system call)`struct sigpending``pending`

Data structure storing the private pending signals

`unsigned long``sas_ss_sp`

Address of alternative signal handler stack

`size_t``sas_ss_size`

Size of alternative signal handler stack

`int (*) (void *)``notifier`

Pointer to a function used by a device driver to block some signals of the process

`void *`

`notifier_data`

Pointer to data that might be used by the notifier function (previous field of table)

`sigset_t *`

`notifier_mask`

Bit mask of signals blocked by a device driver through a notifier function

The `blocked` field stores the signals currently masked out by the process. It is a `sigset_t` array of bits, one for each signal type:

```
typedef struct {
    unsigned long sig[2];
} sigset_t;
```

Because each `unsigned long` number consists of 32 bits, the maximum number of signals that may be declared in Linux is 64 (the `_NSIG` macro specifies this value). No signal can have number 0, so the signal number corresponds to the index of the corresponding bit in a `sigset_t` variable plus one. Numbers between 1 and 31 correspond to the signals listed in [Table 11-1](#), while numbers between 32 and 64 correspond to real-time signals.

11.1.3.1. The signal descriptor and the signal handler descriptor

The `signal` field of the process descriptor points to a signal descriptor, a `signal_struct` structure that keeps track of the shared pending signals. Actually, the signal descriptor also includes fields not strictly related to signal handling, such as the `rlim` per-process resource limit array (see the section "[Process Resource Limits](#)" in [Chapter 3](#)), or the `pgrp` and `session` fields, which store the PIDs of the group leader and of the session leader of the process, respectively (see the section "[Relationships Among Processes](#)" in [Chapter 3](#)). In fact, as mentioned in the section "The `clone()`, `fork()`, and `vfork()` System Calls" in [Chapter 3](#), the signal descriptor is shared by all processes belonging to the same thread group that is, all processes created by invoking the `clone()` system call with the `CLONE_THREAD` flag set thus the signal descriptor includes the fields that must be identical for every process in the same thread group.

The fields of a signal descriptor somewhat related to signal handling are shown in [Table 11-4](#).

Table 11-4. The fields of the signal descriptor related to signal handling

Type

Name

Description

10

`atomic_t`

`count`

Usage counter of the signal descriptor

`atomic_t`

`live`

Number of live processes in the thread group

`wait_queue_head_t`

`wait_chldexit`

Wait queue for the processes sleeping in a `wait4( )` system call

`struct task_struct *`

`curr_target`

Descriptor of the last process in the thread group that received a signal

`struct sigpending`

`shared_pending`

Data structure storing the shared pending signals

`int`

`group_exit_code`

Process termination code for the thread group

`struct task_struct *`

`group_exit_task`

Used when killing a whole thread group

`int`

`notify_count`

Used when killing a whole thread group

`int`

`group_stop_count`

Used when stopping a whole thread group

`unsigned int`

`flags`

Flags used when delivering signals that modify the status of the process

Besides the signal descriptor, every process refers also to a signal handler descriptor, which is a `sigand_struct` structure describing how each signal must be handled by the thread group. Its fields are shown in [Table 11-5](#).

Table 11-5. The fields of the signal handler descriptor

Type

Name

Description

`atomic_t`

`count`

Usage counter of the signal handler descriptor

`struct k_sigaction [64]`

`action`

Array of structures specifying the actions to be performed upon delivering the signals

`spinlock_t`

`siglock`

Spin lock protecting both the signal descriptor and the signal handler descriptor

As mentioned in the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" in [Chapter 3](#), the signal handler descriptor may be shared by several processes by invoking the `clone( )` system call with the `CLONE_SIGHAND` flag set; the `count` field in this descriptor specifies the number of processes that share the structure. In a POSIX multithreaded application, all lightweight processes in the thread group refer to the same signal descriptor and to the same signal handler descriptor.

11.1.3.2. The sigaction data structure

Some architectures assign properties to a signal that are visible only to the kernel. Thus, the properties of a signal are stored in a `k_sigaction` structure, which contains both the properties hidden from the User Mode process and the more familiar `sigaction` structure that holds all the properties a User Mode process

can see. Actually, on the 80 x 86 platform, all signal properties are visible to User Mode processes. Thus the `k_sigaction` structure simply reduces to a single `sa` structure of type `sigaction`, which includes the following fields:^[*]

[*] The `sigaction` structure used by User Mode applications to pass parameters to the `signal( )` and `sigaction( )` system calls is slightly different from the structure used by the kernel, although it stores essentially the same information.

`sa_handler`

This field specifies the type of action to be performed; its value can be a pointer to the signal handler, `SIG_DFL` (that is, the value 0) to specify that the default action is performed, or `SIG_IGN` (that is, the value 1) to specify that the signal is ignored.

`sa_flags`

This set of flags specifies how the signal must be handled; some of them are listed in [Table 11-6](#).^[†]

[†] For historical reasons, these flags have the same prefix "SA_" as the flags of the `irqaction` descriptor (see [Table 4-7](#) in [Chapter 4](#)); nevertheless there is no relation between the two sets of flags.

`sa_mask`

This `sigset_t` variable specifies the signals to be masked when running the signal handler.

Table 11-6. Flags specifying how to handle a signal

Flag Name

Description

`SA_NOCLDSTOP`

Applies only to `SIGCHLD`; do not send `SIGCHLD` to the parent when the process is stopped

`SA_NOCLDWAIT`

Applies only to `SIGCHLD`; do not create a zombie when the process terminates

`SA_SIGINFO`

Provide additional information to the signal handler (see the later section "[Changing a Signal Action](#)")

`SA_ONSTACK`

Use an alternative stack for the signal handler (see the later section "[Catching the Signal](#)")

`SA_RESTART`

Interrupted system calls are automatically restarted (see the later section "[Reexecution of System Calls](#)")

`SA_NODEFER`, `SA_NOMASK`

Do not mask the signal while executing the signal handler

`SA_RESETHAND,`

`SA_ONESHOT`

Reset to default action after executing the signal handler

11.1.3.3. The pending signal queues

As we have seen in [Table 11-2](#) earlier in the chapter, there are several system calls that can generate a signal: some of them `kill( )` and `rt_sigqueueinfo( )` send a signal to a whole thread group, while `otherstkill( )` and `tgkill( )` send a signal to a specific process.

Thus, in order to keep track of what signals are currently pending, the kernel associates two pending signal queues to each process:

- The shared pending signal queue, rooted at the `shared_pending` field of the signal descriptor, stores the pending signals of the whole thread group.
- The private pending signal queue, rooted at the `pending` field of the process descriptor, stores the pending signals of the specific (lightweight) process.

A pending signal queue consists of a `sigpending` data structure, which is defined as follows:

```
struct sigpending {
    struct list_head list;
    sigset_t signal;
}
```

The `signal` field is a bit mask specifying the pending signals, while the `list` field is the head of a doubly linked list containing `sigqueue` data structures; the fields of this structure are shown in [Table 11-7](#).

Table 11-7. The fields of the `sigqueue` data structure

Type

Name

Description

`struct list_head`

`list`

Links for the pending signal queue's list

`spinlock_t *`

lock

Pointer to the siglock field in the signal handler descriptor corresponding to the pending signal

int

flags

Flags of the sigqueue data structure

siginfo_t

info

Describes the event that raised the signal

struct

user_struct *

user

Pointer to the per-user data structure of the process's owner (see the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" in [Chapter 3](#))

The siginfo_t data structure is a 128-byte data structure that stores information about an occurrence of a specific signal; it includes the following fields:

si_signo

The signal number

si_errno

The error code of the instruction that caused the signal to be raised, or 0 if there was no error

si_code

A code identifying who raised the signal (see [Table 11-8](#))

Table 11-8. The most significant signal sender codes

Code Name

Sender

SI_USER

kill() and raise() (see the later section "[System Calls Related to Signal Handling](#)")

SI_KERNEL

Generic kernel function

SI_QUEUE

sigqueue() (see the later section "[System Calls Related to Signal Handling](#)")

SI_TIMER

Timer expiration

SI_ASYNCIO

Asynchronous I/O completion

SI_TKILL

tkill() and tkill() (see the later section "[System Calls Related to Signal Handling](#)")

_sifields

A union storing information depending on the type of signal. For instance, the `siginfo_t` data structure relative to an occurrence of the `SIGKILL` signal records the PID and the UID of the sender process here; conversely, the data structure relative to an occurrence of the `SIGSEGV` signal stores the memory address whose access caused the signal to be raised.

11.1.4. Operations on Signal Data Structures

Several functions and macros are used by the kernel to handle signals. In the following description, `set` is a pointer to a `sigset_t` variable, `nsig` is the number of a signal, and `mask` is an unsigned long bit mask.

sigemptyset(set) and sigfillset(set)

Sets the bits in the `sigset_t` variable to 0 or 1, respectively.

sigaddset(set,nsig) and sigdelset(set,nsig)

Sets the bit of the `sigset_t` variable corresponding to signal `nsig` to 1 or 0, respectively. In practice, `sigaddset( )` reduces to:

```
set->sig[(nsig - 1) / 32] |= 1UL << ((nsig - 1) % 32);
```

and `sigdelset( )` to:

```
set->sig[(nsig - 1) / 32] &= ~(1UL << ((nsig - 1) % 32));
```

`sigaddsetmask(set,mask)` and `sigdelsetmask(set,mask)`

Sets all the bits of the `sigset_t` variable whose corresponding bits of `mask` are on 1 or 0, respectively. They can be used only with signals that are between 1 and 32. The corresponding functions reduce to:

```
set->sig[0] |= mask;
```

and to:

```
set->sig[0] &= ~mask;
```

`sigismember(set,nsig)`

Returns the value of the bit of the `sigset_t` variable corresponding to the signal `nsig`. In practice, this function reduces to:

```
return 1 & (set->sig[(nsig-1) / 32] >> ((nsig-1) % 32));
```

`sigmask(nsig)`

Yields the bit index of the signal `nsig`. In other words, if the kernel needs to set, clear, or test a bit in an element of `sigset_t` that corresponds to a particular signal, it can derive the proper bit through this macro.

`sigandsets(d,s1,s2)`, `sigorsets(d,s1,s2)`, and `signandsets(d,s1,s2)`

Performs a logical AND, a logical OR, and a logical NAND, respectively, between the `sigset_t` variables to which `s1` and `s2` point; the result is stored in the `sigset_t` variable to which `d` points.

`sigtestsetmask(set,mask)`

Returns the value 1 if any of the bits in the `sigset_t` variable that correspond to the bits set to 1 in `mask` is set; it returns 0 otherwise. It can be used only with signals that have a number between 1 and 32.

`siginitset(set,mask)`

Initializes the low bits of the `sigset_t` variable corresponding to signals between 1 and 32 with the bits contained in `mask`, and clears the bits corresponding to signals between 33 and 63.

`siginitsetinv(set,mask)`

Initializes the low bits of the `sigset_t` variable corresponding to signals between 1 and 32 with the complement of the bits contained in `mask`, and sets the bits corresponding to signals between 33 and 63.

`signal_pending(p)`

Returns the value 1 (true) if the process identified by the `*p` process descriptor has nonblocked pending signals, and returns the value 0 (false) if it doesn't. The function is implemented as a simple check on the `TIF_SIGPENDING` flag of the process.

`recalc_sigpending_tsk(t)` and `recalc_sigpending( )`

The first function checks whether there are pending signals either for the process identified by the process descriptor at `*t` (by looking at the `t->pending->signal` field) or for the thread group to which the process belongs (by looking at the `t->signal->shared_pending->signal` field). The function then sets accordingly the `TIF_SIGPENDING` flag in `t->thread_info->flags`. The `recalc_sigpending( )` function is equivalent to `recalc_sigpending_tsk(current)`.

`rm_from_queue(mask,q)`

Removes from the pending signal queue `q` the pending signals corresponding to the bit mask `mask`.

`flush_sigqueue(q)`

Removes from the pending signal queue `q` all pending signals.

`flush_signals(t)`

Deletes all signals sent to the process identified by the process descriptor at `*t`. This is done by clearing the `TIF_SIGPENDING` flag in `t->thread_info->flags` and invoking twice `flush_sigqueue( )` on the `t->pending` and `t->signal->shared_pending` queues.

11.2. Generating a Signal

Many kernel functions generate signals: they accomplish the first phase of signal handlingdescribed earlier in the section "[The Role of Signals](#)"by updating one or more process descriptors as needed. They do not directly perform the second phase of delivering the signal but, depending on the type of signal and the state of the destination processes, may wake up some processes and force them to receive the signal.

When a signal is sent to a process, either from the kernel or from another process, the kernel generates it by invoking one of the functions listed in [Table 11-9](#).

Table 11-9. Kernel functions that generate a signal for a process

Name	Description
<code>send_sig()</code>	Sends a signal to a single process
<code>send_sig_info()</code>	Like <code>send_sig()</code> , with extended information in a <code>siginfo_t</code> structure
<code>force_sig()</code>	Sends a signal that cannot be explicitly ignored or blocked by the process
<code>force_sig_info()</code>	Like <code>force_sig()</code> , with extended information in a <code>siginfo_t</code> structure
<code>force_sig_specific()</code>	Like <code>force_sig()</code> , but optimized for SIGSTOP and SIGKILL signals
<code>sys_tkill()</code>	System call handler of <code>tkill()</code> (see the later section " System Calls Related to Signal Handling ")
<code>sys_tgkill()</code>	System call handler of <code>tgkill()</code>

All functions in [Table 11-9](#) end up invoking the `specific_send_sig_info( )` function described in the next section.

When a signal is sent to a whole thread group, either from the kernel or from another process, the kernel generates it by invoking one of the functions listed in [Table 11-10](#).

Table 11-10. Kernel functions that generate a signal for a thread group

Name

Description

`send_group_sig_info( )`

Sends a signal to a single thread group identified by the process descriptor of one of its members

`kill_pg( )`Sends a signal to all thread groups in a process group (see the section "[Process Management](#)" in [Chapter 1](#))`kill_pg_info( )`Like `kill_pg( )`, with extended information in a `siginfo_t` structure`kill_proc( )`

Sends a signal to a single thread group identified by the PID of one of its members

`kill_proc_info( )`Like `kill_proc( )`, with extended information in a `siginfo_t` structure`sys_kill( )`System call handler of `kill( )` (see the later section "[System Calls Related to Signal Handling](#)")`sys_rt_sigqueueinfo( )`System call handler of `rt_sigqueueinfo( )`

All functions in [Table 11-10](#) end up invoking the `group_send_sig_info( )` function, which is described in the later section "[The group_send_sig_info\(\) Function](#)."

11.2.1. The `specific_send_sig_info( )` Function

The `specific_send_sig_info( )` function sends a signal to a specific process. It acts on three parameters:

sig

The signal number.

info

Either the address of a `siginfo_t` table or one of three special values: 0 means that the signal has been sent by a User Mode process, 1 means that it has been sent by the kernel, and 2 means that it has

been sent by the kernel and the signal is SIGSTOP or SIGKILL.

`t`

A pointer to the descriptor of the destination process.

The `specific_send_sig_info( )` function must be invoked with local interrupts disabled and the `t->sigband->siglock` spin lock already acquired. The function executes the following steps:

1. Checks whether the process ignores the signal; in the affirmative case, returns 0 (signal not generated). The signal is ignored when all three conditions for ignoring a signal are satisfied, that is:
 - ◆ The process is not being traced (PT_PTRACED flag in `t->ptrace` clear).
 - ◆ The signal is not blocked (`sigismember(&t->blocked, sig)` returns 0).
 - ◆ The signal is either explicitly ignored (the `sa_handler` field of `t->sigband->action[sig-1]` is equal to SIG_IGN) or implicitly ignored (the `sa_handler` field is equal to SIG_DFL and the signal is SIGCONT, SIGCHLD, SIGWINCH, or SIGURG).
2. Checks whether the signal is non-real-time (`sig<32`) and another occurrence of the same signal is already pending in the private pending signal queue of the process (`sigismember(&t->pending.signal, sig)` returns 1): in the affirmative case, nothing has to be done, thus returns 0.
3. Invokes `send_signal(sig, info, t, &t->pending)` to add the signal to the set of pending signals of the process; this function is described in detail in the next section.
4. If `send_signal( )` successfully terminated and the signal is not blocked (`sigismember(&t->blocked, sig)` returns 0), invokes the `signal_wake_up( )` function to notify the process about the new pending signal. In turn, this function executes the following steps:
 - a. Sets the TIF_SIGPENDING flags in `t->thread_info->flags`.
 - b. Invokes `try_to_wake_up( )` see the section "[The try_to_wake_up\(\) Function](#)" in [Chapter 7](#) to awake the process if it is either in TASK_INTERRUPTIBLE state, or in TASK_STOPPED state and the signal is SIGKILL.
 - c. If `try_to_wake_up( )` returned 0, the process was already runnable: if so, it checks whether the process is already running on another CPU and, in this case, sends an interprocessor interrupt to that CPU to force a reschedule of the current process (see the section "[Interprocessor Interrupt Handling](#)" in [Chapter 4](#)). Because each process checks the existence of pending signals when returning from the `schedule( )` function, the interprocessor interrupt ensures that the destination process quickly notices the new pending signal.
5. Returns 1 (the signal has been successfully generated).

11.2.2. The `send_signal( )` Function

The `send_signal( )` function inserts a new item in a pending signal queue. It receives as its parameters the signal number `sig`, the address `info` of a `siginfo_t` data structure (or a special code, see the description of `specific_send_sig_info( )` in the previous section), the address `t` of the descriptor of the target process, and the address `signals` of the pending signal queue.

The function executes the following steps:

1. If the value of `info` is 2, the signal is either SIGKILL or SIGSTOP and it has been generated by the kernel via the `force_sig_specific( )` function: in this case, it jumps to step 9. The action

corresponding to these signals is immediately enforced by the kernel, thus the function may skip adding the signal to the pending signal queue.

2. If the number of pending signals of the process's owner (`t->user->sigpending`) is smaller than the current process's resource limit (`t->signal->rlim[RLIMIT_SIGPENDING].rlim_cur`), the function allocates a `sigqueue` data structure for the new occurrence of the signal:

```
q = kmem_cache_alloc(sigqueue_cachep, GFP_ATOMIC);
```

3. If the number of pending signals of the process's owner is too high or the memory allocation in the previous step failed, it jumps to step 9.
4. Increases the number of pending signals of the owner (`t->user->sigpending`) and the reference counter of the per-user data structure pointed to by `t->user`.
5. Adds the `sigqueue` data structure in the pending signal queue signals:

```
list_add_tail(&q->list, &signals->list);
```

6. Fills the `siginfo_t` table inside the `sigqueue` data structure:

```
if ((unsigned long)info == 0) {
    q->info.si_signo = sig;
    q->info.si_errno = 0;
    q->info.si_code = SI_USER;
    q->info._sifields._kill._pid = current->pid;
    q->info._sifields._kill._uid = current->uid;
} else if ((unsigned long)info == 1) {
    q->info.si_signo = sig;
    q->info.si_errno = 0;
    q->info.si_code = SI_KERNEL;
    q->info._sifields._kill._pid = 0;
    q->info._sifields._kill._uid = 0;
} else
    copy_siginfo(&q->info, info);
```

The `copy_siginfo( )` function copies the `siginfo_t` table passed by the caller.

7. Sets the bit corresponding to the signal in the bit mask of the queue:

```
sigaddset(&signals->signal, sig);
```

8. Returns 0: the signal has been successfully appended to the pending signal queue.
9. Here, an item will not be added to the signal pending queue, because there are already too many pending signals, or there is no free memory for the `sigqueue` data structure, or the signal is immediately enforced by the kernel. If the signal is real-time and was sent through a kernel function that is explicitly required to queue it, the function returns the error code `-EAGAIN`:

```
if (sig>=32 && info && (unsigned long) info != 1 &&
    info->si_code != SI_USER)
    return -EAGAIN;
```

10. Sets the bit corresponding to the signal in the bit mask of the queue:

```
sigaddset(&signals->signal, sig);
```

11. Returns 0: even if the signal has not been appended to the queue, the corresponding bit has been set in the bit mask of pending signals.

It is important to let the destination process receive the signal even if there is no room for the corresponding item in the pending signal queue. Suppose, for instance, that a process is consuming too much memory. The kernel must ensure that the `kill( )` system call succeeds even if there is no free memory; otherwise, the system administrator doesn't have any chance to recover the system by terminating the offending process.

11.2.3. The `group_send_sig_info( )` Function

The `group_send_sig_info( )` function sends a signal to a whole thread group. It acts on three parameters: a signal number `sig`, the address `info` of a `siginfo_t` table or alternatively the special values 0, 1, or 2, as explained in the earlier section "[The specific_send_sig_info\(\) Function](#)" and the address `p` of a process descriptor.

The function essentially executes the following steps:

1. Checks that the parameter `sig` is correct:

```
if (sig < 0 || sig > 64)
    return -EINVAL;
```

2. If the signal is being sent by a User Mode process, it checks whether the operation is allowed. The signal is delivered only if at least one of the following conditions holds:

- ◆ The owner of the sending process has the proper capability (usually, this simply means the signal was issued by the system administrator; see [Chapter 20](#)).
- ◆ The signal is `SIGCONT` and the destination process is in the same login session of the sending process.
- ◆ Both processes belong to the same user.

If the User Mode process is not allowed to send the signal, the function returns the value `-EPERM`.

3. If the `sig` parameter has the value 0, it returns immediately without generating any signal:

```
if (!sig || !p->sigband)
    return 0;
```

Because 0 is not a valid signal number, it is used to allow the sending process to check whether it has the required privileges to send a signal to the destination thread group. The function also returns if the destination process is being killed, indicated by checking whether its signal handler descriptor has been released.

4. Acquires the `p->sigband->siglock` spin lock and disables local interrupts.
5. Invokes the `handle_stop_signal( )` function, which checks for some types of signals that might nullify other pending signals for the destination thread group. The latter function executes the following steps:

- a. If the thread group is being killed (`SIGNAL_GROUP_EXIT` flag in the `flags` field of the signal descriptor set), it returns.

- b. If `sig` is a `SIGSTOP`, `SIGTSTP`, `SIGTTIN`, or `SIGTTOU` signal, the function invokes the `rm_from_queue( )` function to remove the `SIGCONT` signal from the shared pending signal queue `p->signal->shared_pending` and from the private queues of all members of the thread group.
- c. If `sig` is `SIGCONT`, it invokes the `rm_from_queue( )` function to remove any `SIGSTOP`, `SIGTSTP`, `SIGTTIN`, and `SIGTTOU` signal from the shared pending signal queue `p->signal->shared_pending`; then, removes the same signals from the private pending signal queues of the processes belonging to the thread group, and awakens them:

```
rm_from_queue(0x003c0000, &p->signal->shared_pending);
t = p;
do {
    rm_from_queue(0x003c0000, &t->pending);
    try_to_wake_up(t, TASK_STOPPED, 0);
    t = next_thread(t);
} while (t != p);
```

The mask `0x003c0000` selects the four stop signals. At each iteration, the `next_thread` macro returns the descriptor address of a different lightweight process in the thread group (see the section "[Relationships Among Processes](#)" in [Chapter 3](#)).^[*]

[*] The actual code is more complicated than the fragment just shown, because `handle_stop_signal( )` also takes care of the unusual case of the `SIGCONT` signal being caught, as well as of the race conditions due to a `SIGCONT` signal occurring while all processes in the thread group are being stopped.

6. Checks whether the thread group ignores the signal; if so, returns the value 0 (success). The signal is ignored when all three conditions for ignoring a signal that are mentioned in the earlier section "[The Role of Signals](#)" are satisfied (see also step 1 in the earlier section "[The specific_send_sig_info\(\) Function](#)").
7. Checks whether the signal is non-real-time and another occurrence of the same signal is already pending in the shared pending signal queue of the thread group: if so, nothing has to be done, thus returns the value 0 (success):

```
if (sig<32 && sigismember(&p->signal->shared_pending.signal,sig))
    return 0;
```

8. Invokes `send_signal( )` to append the signal to the shared pending signal queue (see the previous section "[The send_signal\(\) Function](#)"). If `send_signal( )` returns a nonzero error code, it terminates while returning the same value.
9. Invokes the `_ _group_complete_signal( )` function to wake up one lightweight process in the thread group (see below).
10. Releases the `p->sigband->siglock` spin lock and enables local interrupts.
11. Returns 0 (success).

The `_ _group_complete_signal( )` function scans the processes in the thread group, looking for a process that can receive the new signal. A process may be selected if it satisfies all the following conditions:

- The process does not block the signal.
- The process is not in state `EXIT_ZOMBIE`, `EXIT_DEAD`, `TASK_TRACED`, or `TASK_STOPPED` (as an exception, the process can be in the `TASK_TRACED` or `TASK_STOPPED` states if the signal is `SIGKILL`).
- The process is not being killed that is, its `PF_EXITING` flag is not set.

- Either the process is currently in execution on a CPU, or its `TIF_SIGPENDING` flag is not already set. (In fact, there is no point in awakening a process that has pending signals: in general, this operation has been already performed by the kernel control path that set the `TIF_SIGPENDING` flag. On the other hand, if a process is currently in execution, it should be notified of the new pending signal.)

A thread group might include many processes that satisfy the above conditions. The function selects one of them as follows:

- If the process identified by `p` the descriptor address passed as parameter of the `group_send_sig_info( )` function satisfies all the prior rules and can thus receive the signal, the function selects it.
- Otherwise, the function searches for a suitable process by scanning the members of the thread group, starting from the process that received the last thread group's signal (`p->signal->curr_target`).

If `_group_complete_signal( )` succeeds in finding a suitable process, it sets up the delivery of the signal to the selected process. First, the function checks whether the signal is fatal: in this case, the whole thread group is killed by sending `SIGKILL` signals to each lightweight process in the group. Otherwise, if the signal is not fatal, the function invokes the `signal_wake_up( )` function to notify the selected process that it has a new pending signal (see step 4 in the earlier section "[The specific_send_sig_info\(\) Function](#)").

11.3. Delivering a Signal

We assume that the kernel noticed the arrival of a signal and invoked one of the functions mentioned in the previous sections to prepare the process descriptor of the process that is supposed to receive the signal. But in case that process was not running on the CPU at that moment, the kernel deferred the task of delivering the signal. We now turn to the activities that the kernel performs to ensure that pending signals of a process are handled.

As mentioned in the section "[Returning from Interrupts and Exceptions](#)" in [Chapter 4](#), the kernel checks the value of the `TIF_SIGPENDING` flag of the process before allowing the process to resume its execution in User Mode. Thus, the kernel checks for the existence of pending signals every time it finishes handling an interrupt or an exception.

To handle the nonblocked pending signals, the kernel invokes the `do_signal ( )` function, which receives two parameters:

`regs`

The address of the stack area where the User Mode register contents of the current process are saved.

`oldset`

The address of a variable where the function is supposed to save the bit mask array of blocked signals. It is `NULL` if there is no need to save the bit mask array.

Our description of the `do_signal ( )` function will focus on the general mechanism of signal delivery; the actual code is burdened with lots of details dealing with race conditions and other special cases such as freezing the system, generating core dumps, stopping and killing a whole thread group, and so on. We will quietly skip all these details.

As already mentioned, the `do_signal ( )` function is usually only invoked when the CPU is going to return in User Mode. For that reason, if an interrupt handler invokes `do_signal ( )`, the function simply returns:

```
if ((regs->xcs & 3) != 3)
    return 1;
```

If the `oldset` parameter is `NULL`, the function initializes it with the address of the `current->blocked` field:

```
if (!oldset)
    oldset = &current->blocked;
```

The heart of the `do_signal( )` function consists of a loop that repeatedly invokes the `dequeue_signal( )` function until no nonblocked pending signals are left in both the private and shared pending signal queues. The return code of `dequeue_signal( )` is stored in the `signr` local variable. If its value is 0, it means that all pending signals have been handled and `do_signal( )` can finish. As long as a nonzero value is returned, a pending signal is waiting to be handled. `dequeue_signal( )` is invoked again after `do_signal( )` handles the current signal.

The `dequeue_signal( )` considers first all signals in the private pending signal queue, starting from the lowest-numbered signal, then the signals in the shared queue. It updates the data structures to indicate that the signal is no longer pending and returns its number. This task involves clearing the corresponding bit in `current->pending.signal` or `current->signal->shared_pending.signal`, and invoking `recalc_sigpending( )` to update the value of the `TIF_SIGPENDING` flag.

Let's see how the `do_signal( )` function handles each pending signal whose number is returned by `dequeue_signal( )`. First, it checks whether the current receiver process is being monitored by some other process; in this case, `do_signal( )` invokes `do_notify_parent_cldstop( )` and `schedule( )` to make the monitoring process aware of the signal handling.

Then `do_signal( )` loads the `ka` local variable with the address of the `k_sigaction` data structure of the signal to be handled:

```
ka = &current->sig->action[signr-1];
```

Depending on the contents, three kinds of actions may be performed: ignoring the signal, executing a default action, or executing a signal handler.

When a delivered signal is explicitly ignored, the `do_signal( )` function simply continues with a new execution of the loop and therefore considers another pending signal:

```
if (ka->sa.sa_handler == SIG_IGN)
    continue;
```

In the following two sections we will describe how a default action and a signal handler are executed.

11.3.1. Executing the Default Action for the Signal

If `ka->sa.sa_handler` is equal to `SIG_DFL`, `do_signal( )` must perform the default action of the signal. The only exception comes when the receiving process is `init`, in which case the signal is discarded as described in the earlier section "[Actions](#)".

Performed upon Delivering a Signal":

```
if (current->pid == 1)
    continue;
```

For other processes, the signals whose default action is "ignore" are also easily handled:

```
if (signr==SIGCONT || signr==SIGCHLD ||
    signr==SIGWINCH || signr==SIGURG)
    continue;
```

The signals whose default action is "stop" may stop all processes in the thread group. To do this, `do_signal( )` sets their states to `TASK_STOPPED` and then invokes the `schedule( )` function (see the section "[The `schedule\( \)` Function](#)" in [Chapter 7](#)):

```
if (signr==SIGSTOP || signr==SIGTSTP ||
    signr==SIGTTIN || signr==SIGTTOU) {
    if (signr != SIGSTOP &&
        is_orphaned_pgrp(current->signal->pgrp))
        continue;
    do_signal_stop(signr);
}
```

The difference between `SIGSTOP` and the other signals is subtle: `SIGSTOP` always stops the thread group, while the other signals stop the thread group only if it is not in an "orphaned process group." The POSIX standard specifies that a process group is not orphaned as long as there is a process in the group that has a parent in a different process group but in the same session. Thus, if the parent process dies but the user who started the process is still logged in, the process group is not orphaned.

The `do_signal_stop( )` function checks whether `current` is the first process being stopped in the thread group. If so, it activates a "group stop": essentially, the function sets the `group_stop_count` field in the signal descriptor to a positive value, and awakens each process in the thread group. Each such process, in turn, looks at this field to recognize that a group stop is in progress, changes its state to `TASK_STOPPED`, and invokes `schedule( )`. The `do_signal_stop( )` function also sends a `SIGCHLD` signal to the parent process of the thread group leader, unless the parent has set the `SA_NOCLDSTOP` flag of `SIGCHLD`.

The signals whose default action is "dump" may create a `core` file in the process working directory; this file lists the complete contents of the process's address space and CPU registers. After `do_signal( )` creates the core file, it kills the thread group. The default action of the remaining 18 signals is "terminate," which consists of simply killing the thread group. To kill the whole thread group, the function invokes `do_group_exit( )`, which executes a clean "group exit" procedure (see the section "[Process Termination](#)" in [Chapter 3](#)).

11.3.2. Catching the Signal

If a handler has been established for the signal, the `do_signal( )` function must enforce its execution. It does this by invoking `handle_signal( )`:

```
handle_signal(signr, &info, &ka, oldset, regs);
if (ka->sa.sa_flags & SA_ONESHOT)
    ka->sa.sa_handler = SIG_DFL;
return 1;
```

If the received signal has the `SA_ONESHOT` flag set, it must be reset to its default action, so that further occurrences of the same signal will not trigger again the execution of the signal handler. Notice how `do_signal( )` returns after having handled a single signal. Other pending signals won't be considered until the next invocation of `do_signal( )`. This approach ensures that real-time signals will be dealt with in the proper order.

Executing a signal handler is a rather complex task because of the need to juggle stacks carefully while switching between User Mode and Kernel Mode. We explain exactly what is entailed here:

Signal handlers are functions defined by User Mode processes and included in the User Mode code segment. The `handle_signal( )` function runs in Kernel Mode while signal handlers run in User Mode; this means that the current process must first execute the signal handler in User Mode before being allowed to resume its "normal" execution. Moreover, when the kernel attempts to resume the normal execution of the process, the Kernel Mode stack no longer contains the hardware context of the interrupted program, because the Kernel Mode stack is emptied at every transition from User Mode to Kernel Mode.

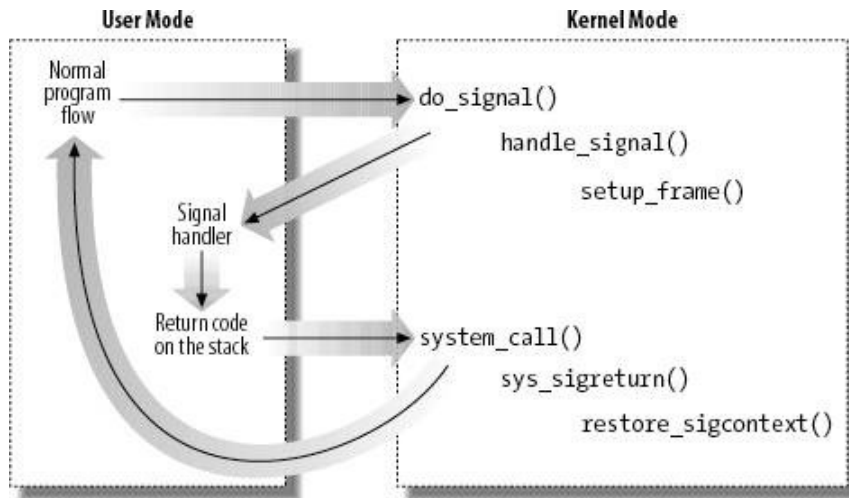
An additional complication is that signal handlers may invoke system calls. In this case, after the service routine executes, control must be returned to the signal handler instead of to the normal flow of code of the interrupted program.

The solution adopted in Linux consists of copying the hardware context saved in the Kernel Mode stack onto the User Mode stack of the current process. The User Mode stack is also modified in such a way that, when the signal handler terminates, the `sigreturn( )` system call is automatically invoked to copy the hardware context back on the Kernel Mode stack and to restore the original content of the User Mode stack.

Figure 11-2 illustrates the flow of execution of the functions involved in catching a signal. A nonblocked signal is sent to a process. When an interrupt or exception occurs, the process switches into Kernel Mode. Right before returning to User Mode, the kernel executes the `do_signal( )` function, which in turn handles the signal (by invoking `handle_signal( )`) and sets up the User Mode stack (by invoking `setup_frame( )` or `setup_rt_frame( )`). When the process switches again to User Mode, it starts executing the signal handler, because the handler's starting address was forced into the program counter. When that function terminates, the return code placed on the User Mode stack by the `setup_frame( )` or `setup_rt_frame( )` function is executed. This code invokes the `sigreturn( )` or the `rt_sigreturn( )` system call; the corresponding service routines copy the hardware context of the

normal program to the Kernel Mode stack and restore the User Mode stack back to its original state (by invoking `restore_sigcontext( )`). When the system call terminates, the normal program can thus resume its execution.

Figure 11-2. Catching a signal



Let's now examine in detail how this scheme is carried out.

11.3.2.1. Setting up the frame

To properly set the User Mode stack of the process, the `handle_signal( )` function invokes either `setup_frame( )` (for signals that do not require a `siginfo_t` table; see the section "[System Calls Related to Signal Handling](#)" later in this chapter) or `setup_rt_frame( )` (for signals that do require a `siginfo_t` table). To choose among these two functions, the kernel checks the value of the `SA_SIGINFO` flag in the `sa_flags` field of the `sigaction` table associated with the signal.

The `setup_frame( )` function receives four parameters, which have the following meanings:

- `sig`
Signal number
- `ka`
Address of the `k_sigaction` table associated with the signal
- `oldset`
Address of a bit mask array of blocked signals

regs

Address in the Kernel Mode stack area where the User Mode register contents are saved

The `setup_frame( )` function pushes onto the User Mode stack a data structure called a frame, which contains the information needed to handle the signal and to ensure the correct return to the `sys_sigreturn( )` function. A frame is a `sigframe` table that includes the following fields (see [Figure 11-3](#)):

pretcode

Return address of the signal handler function; it points to the code at the `__kernel_sigreturn` label (see below).

sig

The signal number; this is the parameter required by the signal handler.

sc

Structure of type `sigcontext` containing the hardware context of the User Mode process right before switching to Kernel Mode (this information is copied from the Kernel Mode stack of `current`). It also contains a bit array that specifies the blocked regular signals of the process.

fpstate

Structure of type `_fpstate` that may be used to store the floating point registers of the User Mode process (see the section "[Saving and Loading the FPU, MMX, and XMM Registers](#)" in [Chapter 3](#)).

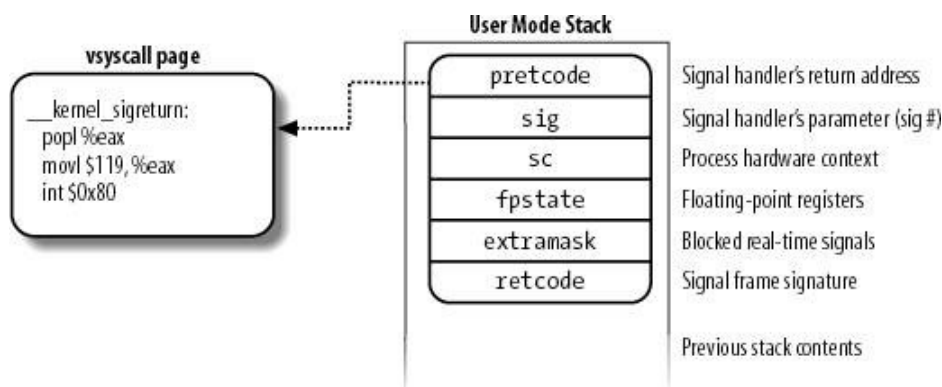
extramask

Bit array that specifies the blocked real-time signals.

retcode

8-byte code issuing a `sigreturn( )` system call. In earlier versions of Linux, this code was effectively executed to return from the signal handler; in Linux 2.6, however, it is used only as a signature, so that debuggers can recognize the signal stack frame.

Figure 11-3. Frame on the User Mode stack



The `setup_frame( )` function starts by invoking `get_sigframe( )` to compute the first memory location of the frame. That memory location is usually^[*] in the User Mode stack, so the function returns the value:

[*] Linux allows processes to specify an alternative stack for their signal handlers by invoking the `signalstack( )` system call; this feature is also required by the X/Open standard. When an alternative stack is present, the `get_sigframe( )` function returns an address inside that stack. We don't discuss this feature further, because it is conceptually similar to regular signal handling.

```
(regs->esp - sizeof(struct sigframe)) & 0xffffffff8
```

Because stacks grow toward lower addresses, the initial address of the frame is obtained by subtracting its size from the address of the current stack top and aligning the result to a multiple of 8.

The returned address is then verified by means of the `access_ok` macro; if it is valid, the function repeatedly invokes `__put_user( )` to fill all the fields of the frame. The `pretcode` field in the frame is initialized to `&_kernel_sigreturn`, the address of some glue code placed in the vsyscall page (see the section "[Issuing a System Call via the sysenter Instruction](#)" in [Chapter 10](#)).

Once this is done, the function modifies the `regs` area of the Kernel Mode stack, thus ensuring that control is transferred to the signal handler when `current` resumes its execution in User Mode:

```

regs->esp = (unsigned long) frame;
regs->eip = (unsigned long) ka->sa.sa_handler;
regs->eax = (unsigned long) sig;
regs->edx = regs->ecx = 0;
regs->xds = regs->xes = regs->xss = __USER_DS;
regs->xcs = __USER_CS;

```

The `setup_frame( )` function terminates by resetting the segmentation registers saved on the Kernel Mode stack to their default value. Now the information needed by the signal handler is on the top of the User Mode stack.

The `setup_rt_frame( )` function is similar to `setup_frame( )`, but it puts on the User Mode stack an extended frame (stored in the `rt_sigframe` data structure) that also includes the content of the `siginfo_t` table associated with the signal. Moreover, this function sets the `pretcode` field so that it points to the `__kernel_rt_sigreturn` code in the `vsyscall` page.

11.3.2.2. Evaluating the signal flags

After setting up the User Mode stack, the `handle_signal( )` function checks the values of the flags associated with the signal. If the signal does not have the `SA_NODEFER` flag set, the signals in the `sa_mask` field of the `sigaction` table must be blocked during the execution of the signal handler:

```
if (!(ka->sa.sa_flags & SA_NODEFER)) {
    spin_lock_irq(&current->sigband->siglock);
    sigorsets(&current->blocked, &current->blocked, &ka->sa.sa_mask);
    sigaddset(&current->blocked, sig);
    recalc_sigpending(current);
    spin_unlock_irq(&current->sigband->siglock);
}
```

As described earlier, the `recalc_sigpending( )` function checks whether the process has nonblocked pending signals and sets its `TIF_SIGPENDING` flag accordingly.

The function returns then to `do_signal( )`, which also returns immediately.

11.3.2.3. Starting the signal handler

When `do_signal( )` returns, the current process resumes its execution in User Mode. Because of the preparation by `setup_frame( )` described earlier, the `eip` register points to the first instruction of the signal handler, while `esp` points to the first memory location of the frame that has been pushed on top of the User Mode stack. As a result, the signal handler is executed.

11.3.2.4. Terminating the signal handler

When the signal handler terminates, the return address on top of the stack points to the code in the `vsyscall` page referenced by the `pretcode` field of the frame:

```
__kernel_sigreturn:
    popl %eax
    movl $__NR_sigreturn, %eax
    int $0x80
```

Therefore, the signal number (that is, the `sig` field of the frame) is discarded from the stack; the `sigreturn( )` system call is then invoked.

The `sys_sigreturn( )` function computes the address of the `pt_regs` data structure `regs`, which contains the hardware context of the User Mode process (see the section "[Parameter Passing](#)" in [Chapter 10](#)). From the value stored in the `esp` field, it can thus derive and check the frame address inside the User Mode stack:

```
frame = (struct sigframe *) (regs.esp - 8);
if (verify_area(VERIFY_READ, frame, sizeof(*frame)) {
    force_sig(SIGSEGV, current);
    return 0;
}
```

Then the function copies the bit array of signals that were blocked before invoking the signal handler from the `sc` field of the frame to the `blocked` field of `current`. As a result, all signals that have been masked for the execution of the signal handler are unblocked. The `recalc_sigpending( )` function is then invoked.

The `sys_sigreturn( )` function must at this point copy the process hardware context from the `sc` field of the frame to the Kernel Mode stack and remove the frame from the User Mode stack; it performs these two tasks by invoking the `restore_sigcontext( )` function.

If the signal was sent by a system call such as `rt_sigqueueinfo( )` that required a `siginfo_t` table to be associated with the signal, the mechanism is similar. The `pretcode` field of the extended frame points to the `__kernel_rt_sigreturn` code in the `vsyscall` page, which in turn invokes the `rt_sigreturn( )` system call; the corresponding `sys_rt_sigreturn( )` service routine copies the process hardware context from the extended frame to the Kernel Mode stack and restores the original User Mode stack content by removing the extended frame from it.

11.3.3. Reexecution of System Calls

The request associated with a system call cannot always be immediately satisfied by the kernel; when this happens, the process that issued the system call is put in a `TASK_INTERRUPTIBLE` or `TASK_UNINTERRUPTIBLE` state.

If the process is put in a `TASK_INTERRUPTIBLE` state and some other process sends a signal to it, the kernel puts it in the `TASK_RUNNING` state without completing the system call (see the section "[Returning from Interrupts and Exceptions](#)" in [Chapter 4](#)). The signal is delivered to the process while switching back to User Mode. When this happens, the system call service routine does not complete its job, but returns an `EINTR`, `ERESTARTNOHAND`, `ERESTART_RESTARTBLOCK`, `ERESTARTSYS`, or `ERESTARTNOINTR` error code.

In practice, the only error code a User Mode process can get in this situation is `EINTR`, which means that the system call has not been completed. (The application programmer may check this code and decide whether to reissue the system call.) The remaining error codes are used internally by the kernel to specify whether the system call may be

reexecuted automatically after the signal handler termination.

Table 11-11 lists the error codes related to unfinished system calls and their impact for each of the three possible signal actions. The terms that appear in the entries are defined in the following list:

Terminate
The system call will not be automatically reexecuted; the process will resume its execution in User Mode at the instruction following the `int $0x80` or `sysenter` one and the `eax` register will contain the `-EINTR` value.

Reexecute
The kernel forces the User Mode process to reload the `eax` register with the system call number and to reexecute the `int $0x80` or `sysenter` instruction; the process is not aware of the reexecution and the error code is not passed to it.

Depends
The system call is reexecuted only if the `SA_RESTART` flag of the delivered signal is set; otherwise, the system call terminates with a `-EINTR` error code.

Table 11-11. Reexecution of system calls

Signal	Error codes and their impact on system call execution		
	EINTR	ERESTARTSYS	ERESTAR
Action			ERESTAR
Default	Terminate	Reexecute	Reexecute
Ignore	Terminate	Reexecute	Reexecute
Catch	Terminate	Depends	Terminate

^a The `ERESTARTNOHAND` and `ERESTART_RESTARTBLOCK` error codes differ on the mechanism used to restart the system c

When delivering a signal, the kernel must be sure that the process really issued a system call before attempting to reexecute it. This is where the `orig_eax` field of the `regs` hardware context plays a critical role. Let's recall how this field is initialized when the interrupt or exception handler starts:

Interrupt
The field contains the IRQ number associated with the interrupt minus 256 (see the section "[Saving the registers for the interrupt handler](#)" in [Chapter 4](#)).

0x80 exception (also sysenter)
The field contains the system call number (see the section "[Entering and Exiting a System Call](#)" in [Chapter 10](#)).

Other exceptions

The field contains the value -1 (see the section "[Saving the Registers for the Exception Handler](#)" in [Chapter 4](#)).

Therefore, a nonnegative value in the `orig_eax` field means that the signal has woken up a `TASK_INTERRUPTIBLE` process that was sleeping in a system call. The service routine recognizes that the system call was interrupted, and thus returns one of the previously mentioned error codes.

11.3.3.1. Restarting a system call interrupted by a non-caught signal

If the signal is explicitly ignored or if its default action is enforced, `do_signal( )` analyzes the error code of the system call to decide whether the unfinished system call must be automatically reexecuted, as specified in [Table 11-11](#). If the call must be restarted, the function modifies the `regs` hardware context so that, when the process is back in User Mode, `eip` points either to the `int $0x80` instruction or to the `sysenter` instruction, and `eax` contains the system call number:

```
if (regs->orig_eax >= 0) {
    if (regs->eax == -ERESTARTNOHAND || regs->eax == -ERESTARTSYS ||
        regs->eax == -ERESTARTNOINTR) {
        regs->eax = regs->orig_eax;
        regs->eip -= 2;
    }
    if (regs->eax == -ERESTART_RESTARTBLOCK) {
        regs->eax = __NR_restart_syscall;
        regs->eip -= 2;
    }
}
```

The `regs->eax` field is filled with the return code of a system call service routine (see the section "[Entering and Exiting a System Call](#)" in [Chapter 10](#)). Notice that both the `int $0x80` and `sysreturn` instructions are two bytes long so the function subtracts 2 from `eip` in order to set it to the instruction that triggers the system call.

The error code `ERESTART_RESTARTBLOCK` is special, because the `eax` register is set to the number of the `restart_syscall( )` system call; thus, the User Mode process does not restart the same system call that was interrupted by the signal. This error code is only used by time-related system calls that, when restarted, should adjust their User Mode parameters. A typical example is the `nanosleep( )` system call (see the section "[An Application of Dynamic Timers: the nanosleep\(\) System Call](#)" in [Chapter 6](#)): suppose that a process invokes it to pause the execution for 20 milliseconds, and that a signal occurs 10 milliseconds later. If the system call would be restarted as usual, the total delay time would exceed 30 milliseconds.

Instead, the service routine of the `nanosleep( )` system call fills the `restart_block` field in the current's `thread_info` structure with the address of a special service routine to be used when restarting, and returns `-ERESTART_RESTARTBLOCK` if interrupted. The `sys_restart_syscall( )` service routine just executes the special `nanosleep( )`'s service routine, which adjusts the delay to consider the time elapsed between the invocation of the original system call and its restarting.

11.3.3.2. Restarting a system call for a caught signal

If the signal is caught, `handle_signal( )` analyzes the error code and, possibly, the `SA_RESTART` flag of the `sigaction` table to decide whether the unfinished system call must be reexecuted:

```

if (regs->orig_eax >= 0) {
    switch (regs->eax) {
        case -ERESTART_RESTARTBLOCK:
        case -ERESTARTNOHAND:
            regs->eax = -EINTR;
            break;
        case -ERESTARTSYS:
            if (!(ka->sa.sa_flags & SA_RESTART)) {
                regs->eax = -EINTR;
                break;
            }
            /* fallthrough */
        case -ERESTARTNOINTR:
            regs->eax = regs->orig_eax;
            regs->eip -= 2;
    }
}

```

If the system call must be restarted, `handle_signal( )` proceeds exactly as `do_signal( )`; otherwise, it returns an `-EINTR` error code to the User Mode process.

11.4. System Calls Related to Signal Handling

As stated in the introduction of this chapter, programs running in User Mode are allowed to send and receive signals. This means that a set of system calls must be defined to allow these kinds of operations.

Unfortunately, for historical reasons, several system calls exist that serve essentially the same purpose. As a result, some of these system calls are never invoked. For instance, `sys_sigaction( )` and `sys_rt_sigaction( )` are almost identical, so the `sigaction( )` wrapper function included in the C library ends up invoking `sys_rt_sigaction( )` instead of `sys_sigaction( )`. We will describe some of the most significant system calls in the following sections.

11.4.1. The `kill( )` System Call

The `kill(pid, sig)` system call is commonly used to send signals to conventional processes or multithreaded applications; its corresponding service routine is the `sys_kill( )` function. The integer `pid` parameter has several meanings, depending on its numerical value:

`pid > 0`

The `sig` signal is sent to the thread group of the process whose PID is equal to `pid`.

`pid = 0`

The `sig` signal is sent to all thread groups of the processes in the same process group as the calling process.

`pid = -1`

The signal is sent to all processes, except swapper (PID 0), init (PID 1), and `current`.

`pid < -1`

The signal is sent to all thread groups of the processes in the process group `-pid`.

The `sys_kill( )` function sets up a minimal `siginfo_t` table for the signal, and then invokes `kill_something_info( )`:

```
info.si_signo = sig;
info.si_errno = 0;
info.si_code = SI_USER;
info._sifields._kill._pid = current->tgid;
info._sifields._kill._uid = current->uid;
return kill_something_info(sig, &info, pid);
```

The `kill_something_info( )` function, in turn, invokes either `kill_proc_info( )` (to send the signal to a single thread group via `group_send_sig_info( )`), or `kill_pg_info( )` (to scan all processes in the destination process group and invoke `send_sig_info( )` for each of them), or repeatedly

`group_send_sig_info( )` for each process in the system (if `pid` is `-1`).

The `kill( )` system call is able to send every signal, even the so-called real-time signals that have numbers ranging from 32 to 64. However, as we saw in the earlier section "[Generating a Signal](#)," the `kill( )` system call does not ensure that a new element is added to the pending signal queue of the destination process, so multiple instances of pending signals can be lost. Real-time signals should be sent by means of a system call such as `rt_sigqueueinfo( )` (see the later section "[System Calls for Real-Time Signals](#)").

System V and BSD Unix variants also have a `killpg( )` system call, which is able to explicitly send a signal to a group of processes. In Linux, the function is implemented as a library function that uses the `kill( )` system call. Another variation is `raise( )`, which sends a signal to the current process (that is, to the process executing the function). In Linux, `raise( )` is implemented as a library function.

11.4.2. The `tkill( )` and `tgkill( )` System Calls

The `tkill( )` and `tgkill( )` system calls send a signal to a specific process in a thread group. The `pthread_kill( )` function of every POSIX-compliant pthread library invokes either of them to send a signal to a specific lightweight process.

The `tkill( )` system call expects two parameters: the PID `pid` of the process to be signaled and the signal number `sig`. The `sys_tkill( )` service routine fills a `siginfo` table, gets the process descriptor address, makes some permission checks (such as those in step 2 in the section "[The group_send_sig_info\(\) Function](#)"), and invokes `specific_send_sig_info( )` to send the signal.

The `tgkill( )` system call differs from `tkill( )` because it has a third parameter: the thread group ID (`tgid`) of the thread group that includes the process to be signaled. The `sys_tgkill( )` service routine performs exactly the same operations as `sys_tkill( )`, but also checks that the process being signaled actually belongs to the thread group `tgid`. This additional check solves a race condition that occurs when a signal is sent to a process that is being killed: if another multithreaded application is creating lightweight processes fast enough, the signal could be delivered to the wrong process. The `tgkill( )` system call solves the problem, because the thread group ID is never changed during the life span of a multithreaded application.

11.4.3. Changing a Signal Action

The `sigaction(sig, act, oact)` system call allows users to specify an action for a signal; of course, if no signal action is defined, the kernel executes the default action associated with the delivered signal.

The corresponding `sys_sigaction( )` service routine acts on two parameters: the `sig` signal number and the `act` table of type `old_sigaction` that specifies the new action. A third `oact` optional output parameter may be used to get the previous action associated with the signal. (The `old_sigaction` data structure contains the same fields as the `sigaction` structure described in the earlier section "[Data Structures Associated with Signals](#)," but in a different order.)

The function checks first whether the `act` address is valid. Then it fills the `sa_handler`, `sa_flags`, and `sa_mask` fields of a `new_ka` local variable of type `k_sigaction` with the corresponding fields of `*act`:

```
_get_user(new_ka.sa.sa_handler, &act->sa_handler);
_get_user(new_ka.sa.sa_flags, &act->sa_flags);
_get_user(mask, &act->sa_mask);
siginitset(&new_ka.sa.sa_mask, mask);
```

The function invokes `do_sigaction( )` to copy the new `new_ka` table into the entry at the `sig-1` position of `current->sig->action` (the number of the signal is one higher than the position in the array because there is no zero signal):

```
k = &current->sig->action[sig-1];
if (act) {
    *k = *act;
    sigdelsetmask(&k->sa.sa_mask, sigmask(SIGKILL) | sigmask(SIGSTOP));
    if (k->sa.sa_handler == SIG_IGN || (k->sa.sa_handler == SIG_DFL &&
        (sig==SIGCONT || sig==SIGCHLD || sig==SIGWINCH || sig==SIGURG))) {
        rm_from_queue(sigmask(sig), &current->signal->shared_pending);
        t = current;
        do {
            rm_from_queue(sigmask(sig), &current->pending);
            recalc_sigpending_tsk(t);
            t = next_thread(t);
        } while (t != current);
    }
}
```

The POSIX standard requires that setting a signal action to either `SIG_IGN` or `SIG_DFL` when the default action is "ignore" causes every pending signal of the same type to be discarded. Moreover, notice that no matter what the requested masked signals are for the signal handler, `SIGKILL` and `SIGSTOP` are never masked.

The `sigaction( )` system call also allows the user to initialize the `sa_flags` field in the `sigaction` table. We listed the values allowed for this field and the related meanings in [Table 11-6](#) (earlier in this chapter).

Older System V Unix variants offered the `signal( )` system call, which is still widely used by programmers. Recent C libraries implement `signal( )` by means of `rt_sigaction( )`. However, Linux still supports older C libraries and offers the `sys_signal( )` service routine:

```
new_sa.sa.sa_handler = handler;
new_sa.sa.sa_flags = SA_ONESHOT | SA_NOMASK;
ret = do_sigaction(sig, &new_sa, &old_sa);
return ret ? ret : (unsigned long)old_sa.sa.sa_handler;
```

11.4.4. Examining the Pending Blocked Signals

The `sigpending( )` system call allows a process to examine the set of pending blocked signals i.e., those that have been raised while blocked. The corresponding `sys_sigpending( )` service routine acts on a single parameter, `set`, namely, the address of a user variable where the array of bits must be copied:

```
sigorsets(&pending, &current->pending.signal,
          &current->signal->shared_pending.signal);
sigandsets(&pending, &current->blocked, &pending);
copy_to_user(set, &pending, 4);
```

11.4.5. Modifying the Set of Blocked Signals

The `sigprocmask( )` system call allows processes to modify the set of blocked signals; it applies only to regular (non-real-time) signals. The corresponding `sys_sigprocmask( )` service routine acts on three parameters:

`o``set`

Pointer in the process address space to a bit array where the previous bit mask must be stored.

`s``et`

Pointer in the process address space to the bit array containing the new bit mask.

`h``ow`

Flag that may have one of the following values:

`SIG_BLOCK`

The `*set` bit mask array specifies the signals that must be added to the bit mask array of blocked signals.

`SIG_UNBLOCK`

The `*set` bit mask array specifies the signals that must be removed from the bit mask array of blocked signals.

`SIG_SETMASK`

The `*set` bit mask array specifies the new bit mask array of blocked signals.

The function invokes `copy_from_user( )` to copy the value pointed to by the `set` parameter into the `new_set` local variable and copies the bit mask array of standard blocked signals of `current` into the `old_set` local variable. It then acts as the `how` flag specifies on these two variables:

```
if (copy_from_user(&new_set, set, sizeof(*set)))
    return -EFAULT;
new_set &= ~(sigmask(SIGKILL) | sigmask(SIGSTOP));
old_set = current->blocked.sig[0];
if (how == SIG_BLOCK)
    sigaddsetmask(&current->blocked, new_set);
else if (how == SIG_UNBLOCK)
    sigdelsetmask(&current->blocked, new_set);
else if (how == SIG_SETMASK)
    current->blocked.sig[0] = new_set;
else
    return -EINVAL;
recalc_sigpending(current);
if (o
```

```

        return -EFAULT;
    return 0;

```

11.4.6. Suspending the Process

The `sigsuspend( )` system call puts the process in the `TASK_INTERRUPTIBLE` state, after having blocked the standard signals specified by a bit mask array to which the `mask` parameter points. The process will wake up only when a nonignored, nonblocked signal is sent to it.

The corresponding `sys_sigsuspend( )` service routine executes these statements:

```

mask &= ~(sigmask(SIGKILL) | sigmask(SIGSTOP));
saveset = current->blocked;
siginitset(&current->blocked, mask);
recalc_sigpending(current);
regs->eax = -EINTR;
while (1) {
    current->state = TASK_INTERRUPTIBLE;
    schedule( );
    if (do_signal(regs, &saveset))
        return -EINTR;
}

```

The `schedule( )` function selects another process to run. When the process that issued the `sigsuspend( )` system call is executed again, `sys_sigsuspend( )` invokes the `do_signal( )` function to deliver the signal that has awakened the process. If that function returns the value 1, the signal is not ignored. Therefore the system call terminates by returning the error code `-EINTR`.

The `sigsuspend( )` system call may appear redundant, because the combined execution of `sigprocmask( )` and `sleep( )` apparently yields the same result. But this is not true: because processes can be interleaved at any time, one must be conscious that invoking a system call to perform action A followed by another system call to perform action B is not equivalent to invoking a single system call that performs action A and then action B.

In this particular case, `sigprocmask( )` might unblock a signal that is delivered before invoking `sleep( )`. If this happens, the process might remain in a `TASK_INTERRUPTIBLE` state forever, waiting for the signal that was already delivered. On the other hand, the `sigsuspend( )` system call does not allow signals to be sent after unblocking and before the `schedule( )` invocation, because other processes cannot grab the CPU during that time interval.

11.4.7. System Calls for Real-Time Signals

Because the system calls previously examined apply only to standard signals, additional system calls must be introduced to allow User Mode processes to handle real-time signals.

Several system calls for real-time signals (`rt_sigaction( )`, `rt_sigpending( )`, `rt_sigprocmask( )`, and `rt_sigsuspend( )`) are similar to those described earlier and won't be discussed further. For the same reason, we won't discuss two other system calls that deal with queues of real-time signals:

rt_sigqueueinfo()

Sends a real-time signal so that it is added to the shared pending signal queue of the destination process. Usually invoked through the `sigqueue( )` standard library function.

rt_sigtimedwait()

Dequeues a blocked pending signal without delivering it and returns the signal number to the caller; if no blocked signal is pending, suspends the current process for a fixed amount of time. Usually invoked through the `sigwaitinfo( )` and `sigtimedwait( )` standard library functions.